

A Borrow-Aware Compiler from MIRLite to OSEA-IR

ANONYMOUS AUTHOR(S)

MIRLite gives typed Rust-like places an operational semantics. Its compiler lowers each statement to OSEA-IR with explicit reads, retags, writes, and borrow retirement. We present both languages through their syntax and transition relations, derive the compiler on a running nested-field assignment, and state the forward simulation that relates source and target memory, locals, and per-cell permission stacks.

Additional Key Words and Phrases: borrow semantics, compiler correctness, Lean, Stacked Borrows

1 Overview and running example

The compiler in this paper connects two views of the same memory operation. MIRLite says *which typed place* a Rust-like statement accesses. OSEA-IR says *which reads, retags, writes, and borrow retirements* implement that access. The proof relates their executions without demanding that their permission stacks be literally equal: the target is allowed to introduce short-lived tags that have no source counterpart. Section 4 names them route tags.

We use cells as the unit of layout. Write $|\tau|$ for the number of cells in a layout τ . Naturals and pointers occupy one cell; a tuple occupies the sum of its fields. A pointer value

$$\text{ptr}(b, o, n, t)$$

denotes address $b + o$ in an allocation with base b , extent n , and provenance tag t . A memory μ maps cell addresses to words, pointers, or undefined values. A permission state Π records the per-cell borrow stacks.

1.1 One example through the whole paper

Let

$$x : (\text{Nat}, (\text{Nat}, \text{Nat})) \quad s_x = x.1.0 := 42.$$

Assume the local x owns three cells beginning at address 100:

$$E(x) = (100, t_x) \quad \mu[100..103) = [\text{word}(5), \text{word}(7), \text{word}(9)].$$

The outer field $.1$ begins one cell into x and has width two; its inner field $.0$ begins at offset zero and has width one. The composed path $.1.0$ therefore has offset 1 and selects exactly address 101. Source execution uses t_x directly to replace 7 by 42. The compiler instead emits

```
r_f := borrow_mut r_x offset 1 length 1
store_nat 42 through r_f
die r_f length 1
```

where r_x holds a pointer to x . The first instruction creates a fresh tag u , the second writes through u , and the third retires u . The running proof must show both executions finish with the same observable cells even though only the target took the intermediate retag steps.

READING GUIDE Each of the next four sections gives the relevant syntax, defines the operational or translation judgment, and then instantiates it on $x.1.0 := 42$. The example is not decorative: it fixes every symbol needed to state the compiler and its correctness relation.

2 MIRLite

MIRLite is a typed, sequential language for the memory-level part of Rust that matters to borrow reasoning. Its semantics is parameterized by a *permission model*: an abstract state Π together with operations that validate and record each memory access (Section 2.3). The core semantics uses

four of them, own, read, ref, and useMut; the full executable language additionally uses borrow retirement, deallocation, exposed provenance, and protector-frame operations. The correctness result instantiates the model with per-cell Stacked Borrows.

2.1 Notation

The following metavariables are used throughout the paper. Primed and subscripted variants (Π' , Π_1 , t_x) denote further values of the same sort.

Symbol	Ranges over / meaning
σ, τ	Layout types. By convention σ is the layout of a region being traversed and τ the layout of the region selected or produced. $ \tau $ is the width of τ in cells.
Γ	A typing context: an ordered list of layouts, one per local.
ℓ	A typed local: an index into Γ together with its layout $\ell : \tau$.
q	A typed projection path $q : \sigma \rightarrow \tau$ with cell offset $\text{off}(q)$.
p, d	Places; d is used when the place is an assignment destination.
e, s	A MIRLite expression and statement.
k, c, m	Retag kind, protector flag, and interior-mutability mask of a reference formation. The compiler's place-lowering judgment is indexed by the same k .
n	A natural count: a length in cells, a number of elements, or a tuple arity. n_s and n_t are source and target step counts.
w	A machine word.
v	A cell value: source form $\text{undef} \mid \text{word}(w) \mid \text{ptr}(b, o, N, t)$ (Section 2.4) or target form $\text{undef} \mid \text{dat}(w) \mid \text{ptr}(b, o, s, t)$ (Section 3.1). Also a list of cells, with $\text{len}(v)$ its length.
a, b, N, o	Cell address, allocation base, allocation extent, and offset within an allocation.
t, u	Provenance tags; u is used for a freshly minted tag.
$S = (i, E, \mu, \Pi)$	A source state: program counter i , environment E , memory μ , and permission state Π .
Π	A permission state (Section 2.3).
own, read, ref, useMut, die	The permission operations on Π (Section 2.3).
θ	An OSEA-IR runtime type; $ \tau $ erases a layout to one, and $\text{typeSize}(\theta)$ is its width in cells.
$T = (j, R, \mu_t, \Pi_t)$	A target state: program counter j , register file R , memory, and permission state.
r	A target register.
Q	A target program: a partial map from labels to instructions.
$C = (n_r, n_l, K, L)$	A compiler state: next register, next label, emitted code map, and local map.
ρ_a, ρ_t	Partial address and tag renamings from source to target.

2.2 Syntax

A *layout type* describes the shape of a memory region. Metavariables σ and τ range over layouts:
 $\tau, \sigma ::= \text{Nat} \mid \text{Ptr } \tau \mid (\tau_0, \dots, \tau_{n-1})$.

Both letters denote layouts; the choice records a role. When a judgment involves two layouts, σ is the layout of the region being traversed and τ is the layout of the region selected from it or produced by it. So a path $q : \sigma \rightarrow \tau$ walks a σ -shaped region and selects a τ -shaped subregion, and a place $\Gamma \vdash p : \tau$ selects a τ -shaped region. When only one layout is involved it is written τ .

Write $|\tau|$ for the number of cells in a layout. Naturals and pointers each occupy one cell, while a tuple occupies the sum of its fields:

$$|\text{Nat}| = 1 \quad |\text{Ptr } \tau| = 1 \quad |(\tau_0, \dots, \tau_{n-1})| = \sum_{j=0}^{n-1} |\tau_j|.$$

The pointee parameter of $\text{Ptr } \tau$ is therefore statically significant even though all pointer values occupy one cell. A context is an ordered list $\Gamma = [\tau_0, \dots, \tau_{n-1}]$. A typed local $\ell : \tau$ consists of an index $j < n$ together with the fact that $\Gamma_j = \tau$.

Projection paths are also typed. The judgment $q : \sigma \rightarrow \tau$ means that following q through a σ -shaped region selects a τ -shaped region. A path may be empty, or select a tuple field and continue recursively. It determines a cell offset $\text{off}(q)$ and satisfies

$$\text{off}(q) + |\tau| \leq |\sigma|.$$

Places are indexed by the layout they select. Write $\Gamma \vdash p : \tau$ for a well-typed place:

$\frac{\ell : \tau \in \Gamma}{\Gamma \vdash \ell : \tau}$	$\frac{\Gamma \vdash p : \sigma \quad q : \sigma \rightarrow \tau}{\Gamma \vdash p.q : \tau}$	$\frac{\Gamma \vdash p : \text{Ptr } \tau}{\Gamma \vdash \star p : \tau}$
<i>local</i>	<i>projection</i>	<i>dereference</i>

The retag kind k is not limited to Rust references:

$$k ::= \text{shared} \mid \text{mutable} \mid \text{raw-const} \mid \text{raw-mut} \mid \text{two-phase}.$$

`raw-const` and `raw-mut` create read-only and mutable raw-pointer items; `two-phase` creates a reserved mutable borrow. Reference formation also takes a protector flag c and an interior-mutability mask m . A protected fresh tag is registered in the current protector frame; true entries of m mark cells lying inside `UnsafeCell`.

The expressions and statements covered by the correctness theorem are typed by the following signatures, where w ranges over machine words:

<code>const(w)</code>	$: \text{Expr } \Gamma \text{ Nat}$
<code>copy(p)</code>	$: \text{Expr } \Gamma \tau \quad \text{if } \Gamma \vdash p : \tau$
<code>ref(k, c, m, p)</code>	$: \text{Expr } \Gamma(\text{Ptr } \tau) \quad \text{if } \Gamma \vdash p : \tau$
<code>$d := e$</code>	$: \text{Stmt } \Gamma \quad \text{if } \Gamma \vdash d : \tau \quad \text{and } \Gamma \vdash e : \tau$
<code>halt</code>	$: \text{Stmt } \Gamma$

Thus the destination and RHS of an assignment necessarily have the same layout. The complete executable syntax also contains uninitialized values, pointer casts and offsets, slice retags, exposed-provenance conversions, conditional assignment, allocation, deallocation, and protector-frame operations. Those forms use the same state model but lie outside the current forward-simulation theorem.

2.3 The permission interface

A permission state Π records, for every allocated cell, which tags may currently access it. The semantics of both languages manipulates Π only through the following partial operations. Each either returns a new state, or fails; a failure is undefined behavior and aborts the execution.

Operation	Meaning
$\text{own}(\Pi, a, n) = (\Pi', t)$	Initialize the n cells from a as a fresh allocation whose owning tag is the fresh tag t .
$\text{read}(\Pi, a, n, t) = \Pi'$	Validate a read of the n cells from a through t .
$\text{useMut}(\Pi, a, n, t) = \Pi'$	Validate a write of the n cells from a through t .
$\text{ref}(\Pi, a, n, t, k, c, m) = (\Pi', u)$	Retag the n cells from a : derive a fresh tag u from parent t with retag kind k , protector flag c , and interior-mutability mask m .
$\text{die}(\Pi, a, n, t) = \Pi'$	Retire tag t on the n cells from a .
$\text{dealloc}(\Pi, a, n, t) = \Pi'$	Deallocate the n cells from a through t and forget their permissions.
$\text{expose}(\Pi, t) = \Pi'$	Record t as exposed, so that a later wildcard access may resolve to it.
$\text{pushProt}(\Pi) = \Pi', \text{popProt}(\Pi) = \Pi'$	Open a protector frame; close the innermost frame, ending the protection of every tag registered in it.

Tags are drawn from a countable set with a distinguished *wildcard* tag, which stands for an unknown provenance recovered from an exposed integer. Every fresh tag returned by `own` or `ref` is distinct from all earlier ones.

The correctness theorem instantiates this interface with per-cell Stacked Borrows. A state is then

$$\Pi = (\text{stacks}, \text{NextTag}, \text{frames}, \text{exposed}),$$

where `stacks` is a partial map from addresses to *borrow stacks*, `NextTag` is the next fresh tag, `frames` is a list of protector frames, each a list of protected tags, and `exposed` is the list of exposed tags. A borrow stack is a list of *items*,

$$i ::= \text{Own}(t) \mid \text{MutRef}(t) \mid \text{Ref}(t) \mid \text{RawPtr}(m', t) \mid \text{Disabled}(t),$$

with the topmost item first and m' a mutability flag. Each operation acts cell by cell over its range. `own` requires an empty stack and pushes `Own( $t$ )`. `read` locates the item carrying t , disables every mutable-reference item above it, and fails if any of those is protected. `useMut` requires the item carrying t to grant writes, pops everything above it, and fails if a popped item is protected. `ref` depends on k . A mutable retag performs a write through t and pushes `MutRef( $u$ )`. A shared or raw-constant retag performs a read through t and pushes `Ref( $u$ )` or `RawPtr(false,  $u$ )`; on cells the mask marks as interior-mutable it instead inserts `RawPtr(true,  $u$ )` directly above the item carrying t , with no access. A raw-mutable retag performs no access and inserts `RawPtr(true,  $u$ )` above the item carrying t , so sibling raw pointers share one group instead of invalidating each other. A two-phase retag performs a read and inserts `RawPtr(true,  $u$ )` likewise, modelling a reservation that stays writable until activation. If c is set, u is registered in the innermost protector frame. `die` pops the item carrying t if it is on top and is neither the allocation's `Own` item nor protected. `dealloc` requires the item carrying t to grant writes at every cell, fails if any item in one of those stacks is protected, and removes the stacks. Fresh tags are minted from `NextTag` upward, so a tag minted later is numerically larger.

In the mechanization the operations are `sb_own`, `sb_read`, `sb_write`, `sb_ref`, `sb_die`, and `sb_dealloc` in `src/obseq3/sb.lean`; the paper writes `useMut` for `sb_write` to keep the source-level reading.

2.4 State and place semantics

A source *cell value* is undefined, a machine word, or a pointer:

$$v ::= \text{undef} \mid \text{word}(w) \mid \text{ptr}(b, o, N, t).$$

A pointer $\text{ptr}(b, o, N, t)$ denotes address $b + o$ in the allocation with base b and extent N , and carries provenance tag t . A source *memory* μ consists of a partial map from addresses to cell values, the next bump-allocation address, and the list of known allocation ranges. Write $\mu(a)$ for the cell at address a , which is `undef` when the map has no entry there; $\mu[a..a + n]$ for the list of n cells starting at a ; and $\mu[a \mapsto v]$ for the memory with the cell at a replaced by v .

A source state is $S = (i, E, \mu, \Pi)$. The program counter i indexes the source statement list. The partial environment E maps a typed local either to no binding or to an allocation base and tag. Finally, Π is the abstract permission state.

A resolved place $\text{res}(a, t, b, N)$ records current address a , access tag t , allocation base b , and allocation extent N . For a well-typed place $\Gamma \vdash p : \tau$, write

$$S \vdash_a p \Downarrow (\text{res}(a, t, b, N), \Pi')$$

when resolving p for an access succeeds. Resolution does not write memory, but dereferencing a pointer is itself a read and can therefore change the permission state. All rules in this section describe successful branches. An unbound local, a malformed or out-of-bounds pointer, or rejection by the permission model produces an error and therefore no successor judgment.

$\frac{E(\ell) = (a, t)}{S \vdash_a \ell \Downarrow (\text{res}(a, t, a, \tau), \Pi)}$	$\frac{S \vdash_a p \Downarrow (\text{res}(a, t, b, N), \Pi')}{S \vdash_a p.q \Downarrow (\text{res}(a + \text{off}(q), t, b, N), \Pi')}$
<i>local</i>	<i>projection</i>

Projection is only typed address arithmetic: it preserves provenance, allocation bounds, and permissions. Dereference changes provenance because the pointer *value*, not the place holding it, identifies the referent:

$\frac{\begin{array}{l} S \vdash_a p \Downarrow (\text{res}(a, t, b, N), \Pi_1) \\ b \leq a < b + N \quad \text{read}(\Pi_1, a, 1, t) = \Pi_2 \\ \mu(a) = \text{ptr}(b', o', N', t') \end{array}}{S \vdash_a \star p \Downarrow (\text{res}(b' + o', t', b', N'), \Pi_2)}$
<i>dereference</i>

Nested dereferences thread these permission changes from the inside out. A separate *pure lookup* follows the same address calculation but merely inspects a pointer cell: it performs neither a bounds check nor read. It is used for raw observations such as a conditional discriminant and for deciding whether an assignment root must first be allocated. Ordinary reads, retags, and writes use access resolution above.

2.5 Preparation, expressions, and statements

Before an assignment, $\text{prepare}(S, d)$ checks the destination using pure lookup. If lookup succeeds, preparation returns S unchanged. If it fails and the root of d is an unbound local $\ell : \tau$, the bump allocator reserves $|\tau|$ cells at a fresh base b , `own` returns a fresh tag t , and the environment is

extended with $E(\ell) = (b, t)$. An unresolved destination rooted through a dereference is an error: assignment never allocates an implicit pointee.

Write $S \vdash e \Downarrow_{\tau} (v, S')$ when expression e produces a list of exactly $|\tau|$ cells. Constants have an explicit one-cell transition:

$$S \vdash \text{const}(w) \Downarrow_{\text{Nat}} ([\text{word}(w)], S)$$

constant

Copy and reference formation first resolve their source place and then act on the entire selected range. Reading an absent but permitted memory cell yields undef; bounds or permission failure still produces an error.

$$\frac{S \vdash_a p \Downarrow (\text{res}(a, t, b, N), \Pi_1) \quad a + |\tau| \leq b + N \quad \text{read}(\Pi_1, a, |\tau|, t) = \Pi_2}{S \vdash \text{copy } p \Downarrow_{\tau} (\mu[a..a + |\tau|], S[\Pi \rightarrow \Pi_2])}$$

copy

$$\frac{S \vdash_a p \Downarrow (\text{res}(a, t, b, N), \Pi_1) \quad a + |\tau| \leq b + N \quad \text{ref}(\Pi_1, a, |\tau|, t, k, c, m) = (\Pi_2, u)}{S \vdash \text{ref}(k, c, m, p) \Downarrow_{\text{Ptr } \tau} ([\text{ptr}(b, a - b, N, u)], S[\Pi \rightarrow \Pi_2])}$$

reference

Assignment threads the complete states returned by these phases. Write $\text{write}(S, a, t, b, N, v) = S'$ for the operation that checks $a + \text{len}(v) \leq b + N$, applies useMut through t , replaces the selected cells, and increments the source PC. The transition order is:

$$\frac{\begin{array}{l} \text{prepare}(S, d) = S_1 \\ S_1 \vdash e \Downarrow_{\tau} (v, S_2) \\ S_2 \vdash_a d \Downarrow (\text{res}(a, t, b, N), \Pi_3) \\ \text{write}(S_2[\Pi \rightarrow \Pi_3], a, t, b, N, v) = S_3 \end{array}}{S \vdash d := e \rightarrow S_3}$$

assignment

This formulation preserves environment and allocator changes made during preparation and all state changes made by RHS evaluation. Crucially, the whole RHS is evaluated before destination access resolution. Hence $\text{copy}(p)$ materializes its cells before a destination access can invalidate a tag needed by the source.

At the program level, the current PC selects a statement. A successful assignment advances it through `write`; `halt` and a missing statement leave the state fixed. A fuel-bounded run repeatedly performs these steps and stops at the first error or fixed point.

2.6 Running source execution

Take the one-local context

$$\Gamma = [(\text{Nat}, (\text{Nat}, \text{Nat}))]$$

and let x be its index-zero local. Assume x was previously allocated by `own`, so that $E(x) = (100, t_x)$ and t_x is the owning tag for the three-cell block. Let

$$\mu(100) = \text{word}(5) \quad \mu(101) = \text{word}(7) \quad \mu(102) = \text{word}(9).$$

Write q_1 for the path selecting the outer tuple's second field and q_0 for the path selecting that inner tuple's first field. Their types and offsets are

$$\begin{array}{l} q_1 : (\text{Nat}, (\text{Nat}, \text{Nat})) \rightarrow (\text{Nat}, \text{Nat}) \\ q_0 : (\text{Nat}, \text{Nat}) \rightarrow \text{Nat} \quad \text{off}(q_1) = 1 \quad \text{off}(q_0) = 0. \end{array}$$

The surface spelling $x.1.\emptyset$ denotes the nested place $(x.q_1).q_0$; the composed path used later is $q_1.append(q_0)$ and has offset $1 + 0 = 1$. Root preparation is a no-op because x is already bound. The place derivation is explicit:

$$\begin{aligned} S &\vdash_a x \Downarrow (\text{res}(100, t_x, 100, 3), \Pi) \\ S &\vdash_a x.q_1 \Downarrow (\text{res}(101, t_x, 100, 3), \Pi) \\ S &\vdash_a (x.q_1).q_0 \Downarrow (\text{res}(101, t_x, 100, 3), \Pi). \end{aligned}$$

The inner zero-offset projection changes neither the address nor provenance. The constant transition produces $[\text{word}(42)]$. Assume the permission model accepts the one-cell write:

$$\text{useMut}(\Pi, 101, 1, t_x) = \Pi_{s'}.$$

If $S = (i, E, \mu, \Pi)$, the assignment rule therefore yields

$$\begin{aligned} S &\vdash (x.q_1).q_0 := \text{const}(42) \rightarrow \\ &(i + 1, E, \mu[101 \rightarrow \text{word}(42)], \Pi_{s'}). \end{aligned}$$

The visible block is now $[5, 42, 9]$. There is no source retag: both projections are structural, and the write uses t_x directly. Address 102, corresponding to sibling path $.1.1$, is untouched. This single `useMut` event is the behavior the longer target trace must simulate.

Formalization of the typed syntax and source transition system: `src/obseq3/syntax.lean` and `src/obseq3/mirlite_semantics.lean`.

3 OSEA-IR

OSEA-IR is a register machine that exposes the memory and permission events implicit in a MIRLite statement. A source place is gone: an instruction names a register containing a concrete pointer, an offset, and a number of cells. Consequently one source step may require several target steps and may create short-lived tags that have no source counterpart; Section 4 names them route tags and fixes exactly when they are created and retired.

3.1 Values, runtime types, and programs

Target values and runtime types are distinct from MIRLite layouts:

v	$::=$	$\text{undef} \mid \text{dat}(w) \mid \text{ptr}(b, o, s, t)$
θ	$::=$	$\text{NatTy} \mid \text{PTy} \mid \text{TupTy}([\theta_1, \dots, \theta_n])$
R	$::=$	$[r \mapsto (\theta, [v_1, \dots, v_n])]$
Q	$::=$	$\mathbb{N} \mapsto \text{option}(I)$

The pointer fields have the same meaning as at the source: allocation base b , current offset o , allocation size s , and provenance tag t . `dat` is a machine word. `undef` represents an absent or explicitly uninitialized cell. A register entry contains a runtime type and an entire cell list; the register file itself is a finite shadowing map.

The erasure $[\tau]$ converts a source layout to a runtime type:

$$[\text{Nat}] = \text{NatTy} \quad [\text{Ptr } \tau] = \text{PTy} \quad [(\tau_1, \dots, \tau_n)] = \text{TupTy}([\tau_1], \dots, [\tau_n]).$$

Thus all pointer layouts erase to `PTy`: OSEA-IR records that a register holds a pointer, but not the pointee layout. Sizes agree, $\text{typeSize}([\tau]) = |\tau|$. This distinction matters in the rules below: a target load is indexed by θ , not by a source layout τ .

The relevant RHS and instruction syntax is:

```

287       $h \quad ::=$   $\text{load}_{\theta(r)} \mid \text{alloc}_{\theta} \mid \text{allocN}_{\theta(n)}$ 
288               $\mid \text{allocDyn}_{\theta(r)} \mid \text{borrow}(k, c, m, n, r, \delta)$ 
289               $\mid \text{expose}(r) \mid \text{fromExposed}(r) \mid \text{offset}(r, d) \mid \text{borrowRest}(k, c, r)$ 
290       $I \quad ::=$   $r := h \mid \text{store}_{\theta(r_s, r_p)} \mid \text{storec}_{\theta([v_1, \dots, v_n], r_p)}$ 
291               $\mid \text{memcpy}_{\theta(r_d, r_s)} \mid \text{die}(r, n) \mid \text{dealloc}(r)$ 
292               $\mid \text{skipIf}(r, w, n) \mid \text{pushProt} \mid \text{popProt} \mid \text{halt}$ 
293

```

The full syntax is shown because it fixes the executable language boundary. The correctness theorem in the final section covers a smaller core. A target program Q is a partial map from numeric labels to instructions. Partiality lets the compiler extend code without renumbering an earlier fragment.

3.2 Machine state and RHS evaluation

A target state is

$$T = (j, R, \mu_t, \Pi_t).$$

j is the program counter. Target memory maps addresses to target values and also carries a bump-allocation watermark and an allocation table. A missing memory cell reads as `undef`. The permission state is parameterized; the correctness result uses the same per-cell Stacked Borrows instance as MIRLite.

Write

$$T \vdash h \Downarrow (\theta, |v|, T')$$

when evaluating h succeeds with runtime type θ , value cells $|v|$, and state T' . Evaluation may update memory or permissions but does not advance j . The two operations used by the running example are:

$$\begin{array}{c}
 R(r) = (\theta_r, [\text{ptr}(b, o, s, t)]) \\
 a = b + o \quad b \leq a \quad a + \text{typeSize}(\theta) \leq b + s \\
 \text{read}(\Pi_t, a, \text{typeSize}(\theta), t) = \Pi_{t'} \\
 \hline
 (j, R, \mu_t, \Pi_t) \vdash \text{load}_{\theta(r)} \Downarrow \\
 (\theta, \mu_t[a..a+\text{typeSize}(\theta)], (j, R, \mu_t, \Pi_{t'}))
 \end{array}$$

T-Load

$$\begin{array}{c}
 R(r) = (\theta_r, [\text{ptr}(b, o, s, t)]) \\
 a = b + o + \delta \quad a + n \leq b + s \\
 \text{ref}(\Pi_t, a, n, t, k, c, m) = (\Pi_{t'}, u) \\
 \hline
 (j, R, \mu_t, \Pi_t) \vdash \text{borrow}(k, c, m, n, r, \delta) \Downarrow \\
 (\text{PTy}, [\text{ptr}(b, o + \delta, s, u)], (j, R, \mu_t, \Pi_{t'}))
 \end{array}$$

T-Borrow

Both rules require r to contain exactly one pointer; the stored runtime type θ_r is immaterial to pointer extraction. `load` checks both ends of the complete range. `borrow` checks its upper bound; offsets are natural words, so no separate negative-offset case exists. In particular, a zero-length borrow at one past the end is legal and performs a zero-cell permission operation.

Allocation illustrates an RHS that changes both memory and permissions. If the allocator returns fresh base b for $\text{typeSize}(\theta)$ cells and

$$\text{own}(\Pi_t, b, \text{typeSize}(\theta)) = (\Pi_{t'}, u),$$

then `allocθ` returns $([\text{ptr}(b, 0, \text{typeSize}(\theta), u)], \text{PTy})$. `allocN` allocates $n \cdot \text{typeSize}(\theta)$ cells. `allocDyn` first reads a dat value through the pointer in its register, then allocates that many elements. This order matches the source's permission-visible length read.

The remaining pointer RHSs are operational rather than mere casts. `expose` reads a pointer-valued cell and exposes the stored tag; `fromExposed` reads an integer cell and returns a wildcard-tagged pointer; `offset` reads a stored pointer and changes its offset while preserving its tag; and `borrowRest` reads a stored pointer and retags the remaining range from its runtime offset to the

end of its allocation. Wrong register shape, wrong memory value, out-of-bounds arithmetic, or a failed permission call produces an error.

3.3 Instruction steps

We write $Q \vdash T \rightarrow T'$ when fetching and executing one instruction succeeds. Assignment evaluates its RHS before inserting the returned entry:

$$\frac{Q(j) = r := h \quad (j, R, \mu_t, \Pi_t) \vdash h \Downarrow (\theta, |v|, (j, R_1, \mu_1, \Pi_1))}{Q \vdash (j, R, \mu_t, \Pi_t) \rightarrow (j+1, R_1[r \mapsto (\theta, |v|)], \mu_1, \Pi_1)}$$

T-Assgn

Stores share a helper that extracts one pointer $\text{ptr}(b, o, s, t)$ from r_p , checks $b + o + \text{len}(|v|) \leq b + s$, performs $\text{useMut}(\Pi_t, b + o, \text{len}(|v|), t) = \Pi_{t'}$, writes the cells, and advances the PC. The two store forms supply $|v|$ differently:

Instruction	Additional check	Permission and memory effect
$\text{store}_{\theta(r_s, r_p)}$	$R(r_s) = (\theta, v)$.	useMut for $\text{len}(v)$ cells, then write $ v $.
$\text{storec}_{\theta(v , r_p)}$	$\text{len}(v) = \text{typeSize}(\theta)$.	The same write-through-pointer operation.
$\text{memcpy}_{\theta(r_d, r_s)}$	Both registers contain pointers; complete ranges fit and do not overlap.	read the source range, useMut the destination range, then copy.

For $\text{die}(r, n)$, the register must contain one pointer and the machine calls $\text{die}(\Pi_t, b + o, n, t)$. On success only permissions and the PC change. Unlike load, borrow, and store, this instruction performs no separate allocation-range check: its admissibility is exactly that of the permission operation. dealloc additionally requires offset zero, retires the whole allocation, and removes its cells. Protector instructions update the frame stack. skipIf inspects a dat discriminant without a permission event and either advances one label or skips the following n labels.

If $Q(j)$ is missing or is halt , the state is a fixed point. Define $Q \vdash T \xrightarrow{n} T'$ by exactly n iterations of the step function. Iteration does not stop early at a fixed point, so additional fuel simply repeats it; an error aborts immediately. This convention makes the target step count in the simulation existential without requiring a separate reflexive-transitive-closure relation.

3.4 Remaining operational cases

The other RHS forms use the same three-stage pattern: validate the register shape, perform permission events in source order, then construct a result. Writing $R(r) = (\theta_r, [\text{ptr}(b, o, s, t)])$ and $a = b + o$, their successful behavior is:

RHS	Permission-visible work	Returned register entry
$\text{allocN}_{\theta(n)}$	own a fresh $n \cdot \text{typeSize}(\theta)$ -cell block.	PTy and one base pointer spanning the block.
$\text{allocDyn}_{\theta(r)}$	read one cell at a ; require $\text{dat}(n)$; then own a fresh $n \cdot \text{typeSize}(\theta)$ -cell block.	The fresh base pointer at runtime type PTy.
$\text{expose}(r)$	read the pointer cell at a ; require a stored pointer and expose its tag.	NatTy and dat of the stored pointer's current address.

fromExposed(r)	read the integer cell at a ; resolve it against the allocation table.	PTy and a pointer carrying the wildcard tag.
offset(r, d)	read the pointer cell at a ; reject a negative resulting offset.	PTy and the stored pointer with offset increased by d ; preserve its tag.
borrowRest(k, c, r)	read the pointer cell at a ; then ref its remaining $s - o$ cells with empty mask.	PTy and the stored pointer carrying the fresh tag.

Notice that these instructions read *through the outer pointer in the register* before inspecting the value in memory. For example, `offset` does not change the offset of that outer pointer. It loads a pointer-valued cell, changes the loaded value, and returns the result in a register. This is why the compiler can use the same RHS to implement MIRLite pointer arithmetic without suppressing the source read event.

The instruction-only cases have equally concrete state effects:

Instruction	Successful transition
dealloc(r)	Require one pointer with offset zero; call <code>dealloc</code> on its entire allocation, remove those memory cells, and advance. The bump allocator does not reuse the range.
skipIf(r, w, n)	Require one pointer and <code>dat(w')</code> at its current address. If $w' = w$, advance to $j + 1$; otherwise advance to $j + 1 + n$. This inspection intentionally has no permission event, matching the source conditional.
pushProt	Push an empty protector frame and advance.
popProt	Pop the top frame if the permission model permits it, then advance.
halt	Return the input state unchanged.

There is no global well-formedness premise saying every register cell list matches its runtime type. Instead, the instruction that consumes a register checks the shape it needs. A load requires a singleton pointer but ignores the entry’s stored type; store checks source-type equality; storec checks constant length; and pointer-manipulating forms require a pointer-valued memory cell after their read. This design keeps malformed target programs executable with explicit errors while the compiler proves that its own successful outputs meet the required local checks.

3.5 Error boundaries

OSEA-IR errors fall into four semantic classes rather than one generic “stuck” state:

Class	Typical cause	Where detected
Register shape	Missing register, non-pointer singleton, or wrong source runtime type.	Before any memory or permission change.
Value shape	A cast, dynamic allocation, or branch observes the wrong memory-value constructor.	After any required permission read, so that event order remains explicit.
Spatial	A complete access range exceeds its allocation, a pointer offset becomes negative, or <code>memcpy</code> overlaps.	Before the corresponding read/write permission event.

Permission	read, ref, useMut, die, dealloc, or protector pop rejects the operation.	At the abstract permission interface; its error is propagated.
------------	--	--

The forward simulation only starts from a successful source step. It therefore proves that the compiled fragment avoids all of these errors for the covered cases; it does not claim that source and target failure messages coincide.

3.6 Running OSEA-IR execution

Let $\tau_x = (\text{Nat}, (\text{Nat}, \text{Nat}))$ and $\theta_x = \lfloor \tau_x \rfloor$. At entry:

$$T_0 = (j, R_0, \mu_0, \Pi_0) \quad R_0(r_x) = (\text{PTy}, [\text{ptr}(100, 0, 3, \hat{t}_x)]) \\ \mu_0[100..103] = [\text{dat}(5), \text{dat}(7), \text{dat}(9)].$$

The register type is PTy, not the source layout Ptr τ_x . Assume the three permission calls needed by the fragment succeed:

$$\text{ref}(\Pi_0, 101, 1, \hat{t}_x, \text{mutable}, \text{false}, []) = (\Pi_1, u) \\ \text{useMut}(\Pi_1, 101, 1, u) = \Pi_2 \quad \text{die}(\Pi_2, 101, 1, u) = \Pi_3.$$

Install the following code:

$Q(j)$	$r_f := \text{borrow}(\text{mutable}, \text{false}, [], 1, r_x, 1)$
$Q(j+1)$	$\text{storec}_{\text{NatTy}}([\text{dat}(42)], r_f)$
$Q(j+2)$	$\text{die}(r_f, 1)$

The rules determine all intermediate states:

$$T_0 \rightarrow T_1 = (j+1, R_0[r_f \mapsto (\text{PTy}, [\text{ptr}(100, 1, 3, u)])], \mu_0, \Pi_1) \\ \rightarrow T_2 = (j+2, R_1, \mu_0[101 \mapsto \text{dat}(42)], \Pi_2) \\ \rightarrow T_3 = (j+3, R_1, \mu_0[101 \mapsto \text{dat}(42)], \Pi_3).$$

Thus $Q \vdash T_0 \xrightarrow{3} T_3$. The temporary register remains, but its permission item has been retired. The target performs one extra ref and one extra die; its sole memory update is the same one-cell update as the source.

Formalization of target values, RHS evaluation, instruction stepping, and fuelled execution: `src/obseq3/oseair.lean`.

4 Compiler

The compiler is a state-passing translation from typed MIRLite syntax to OSEA-IR code. Its semantic obligation is to preserve source evaluation order while introducing explicit loads, retags, stores, and retirement of route tags (Definition 4.1).

4.1 Compiler state and translation judgments

A compiler state is

$$C = (n_r, n_l, K, L).$$

n_r and n_l are the next fresh register and label. K is the emitted code map, and L maps a source-local index to a pair (r, τ) . The layout in L is a source layout; code uses its runtime erasure. A compiler computation is monotone: counters do not decrease, code below the old n_l is unchanged, and every old local-map entry remains present.

For states $C = (n_r, n_l, K, L)$ and $C' = (n_{r'}, n_{l'}, K', L')$, define their emitted interval

$$\text{new}(C, C') = [K'(n_l), K'(n_l + 1), \dots, K'(n_{l'} - 1)].$$

This makes the translation judgments state what the executable compiler does, without pretending that it separately returns a code list:

$$C \vdash_k p \rightarrow C'; (r, D) \quad C \vdash e \rightarrow C'; (F, D_e) \quad C \vdash s \rightarrow C'.$$

The first judgment lowers place p , leaves its pointer in r , and returns cleanup list D . Its index k is the retag kind of Section 2.2: any route borrow the lowering emits, in the sense of the following definition, uses k . Reads lower their place with $k = \text{shared}$ and assignment destinations with $k = \text{mutable}$.

Definition 4.1 (Route borrow) *A route borrow is a borrow instruction emitted by place lowering to obtain a pointer to a projected subregion of a place that the compiled code already holds a pointer to. Its fresh tag is a route tag. A route borrow is unprotected, has an empty mask, and spans exactly the width of the layout selected by the place. Its register and width are recorded as an entry (r, n) of the cleanup list D , and the tag is retired by $\text{die}(r, n)$ before the next source-statement boundary. A route tag is never stored to memory by the compiled code and has no source counterpart.*

Route tags are the only tags the compiled code mints that the source does not. Every other target retag corresponds to a source ref and produces a tag that the source program also stores. Cleanup reverses D and emits a die for each entry. The expression judgment emits all pre-destination work, returns a store function F waiting for a destination register, and records any post-store cleanup. The statement judgment emits the complete fragment.

Compilation is checked. A local lookup may fail when lowering a read whose root has not been established. Unsupported is retained as a future-facing error, although the current compiler handles every executable source constructor. Successful computations still carry the monotonicity property.

4.2 Establishing local roots

Writing an unbound root must mirror source preparation. If $L(\ell)$ is already defined, root preparation returns its register and emits nothing. If it is absent and $\ell : \tau$, the compiler chooses fresh r , emits

$$r := \text{alloc}_{[\tau]}.$$

and extends L with $\ell \mapsto (r, \tau)$. For a projected destination it recurses to the base, and for a dereference it recurses to the pointer place. This happens before RHS lowering, just as MIRLite allocates an unbound assignment root before evaluating the RHS.

Read lowering does not silently allocate a missing local. This separates two semantic facts: assignment preparation may establish a root; resolving a source read requires that the root already exist.

4.3 Canonical projections and flattening

A place may contain nested projections, but emitting a borrow for each syntactic layer would change the accessed ranges. The canonical operation fuses adjacent paths:

```
def projInto : Place  $\Gamma$   $\sigma \rightarrow \text{PathTo } \sigma \tau \rightarrow \text{Place } \Gamma \tau$ 
  | .proj b q, p => .proj b (q.append p)
  | b,          p => .proj b p

def flattenPlace : Place  $\Gamma \tau \rightarrow \text{Place } \Gamma \tau$ 
  | .local l      => .local l
```

```

494 | .deref p      => .deref (flattenPlace p)
495 | .proj b path => projInto (flattenPlace b) path

```

Here σ and τ are the traversed and selected layouts of a typed path, as in Section 2.2; the code is not manipulating untyped field lists. If $q : \sigma_0 \rightarrow \sigma$ and $p : \sigma \rightarrow \tau$, then

```

498 q.append(p) : \sigma_0 \rightarrow \tau \quad \wedge \quad \text{off}(q.append(p)) = \text{off}(q) + \text{off}(p).

```

Flattening stops at a dereference because an offset cannot be moved across a memory-loaded pointer. It preserves the selected layout and the source resolution result. The compiler implements the same normalization on the fly: when it sees $(b.q).p$, it restarts lowering at $b.(q.append(p))$ before emitting any projection borrow.

503 4.4 Ordinary place lowering

After reassociation, ordinary lowering is defined by the following cases. The selected place has layout τ .

506 Place	Translation	Returned result
507 ℓ	Look up $L(\ell) = (r, \tau)$.	$(r, [])$; no code.
508 $b.q, \text{off}(q) = 0$	Lower b with the same kind k .	Reuse the base register and cleanup.
509 $b.q, \delta = \text{off}(q) > 0$	Lower b to (r_b, D_b) ; choose fresh r_f ; emit 510 $r_f := \text{borrow}(k, \text{false}, [], \tau , r_b, \delta)$.	$(r_f, D_b + [(r_f, \tau)])$.
511 $\star p$	Lower pointer place p shared; load PTy 512 into fresh r ; retire only the route tags 513 used to reach p .	$(r, [])$. The loaded tag came from memory and is not a route tag.

Offset-zero projections reuse the base pointer because they denote the same address and require no narrower pointer. Nonzero projections emit a route borrow (Definition 4.1), and so borrow only the *final selected width*. A dereference loads a pointer cell. It must not retire the loaded pointer: its tag belongs to the source program, whereas cleanup is only for route tags.

For x.1.0, direct recursive lowering without reassociation would first borrow the two-cell field x.1 and then reuse its offset-zero subfield. That would touch addresses 101 and 102. Flattening instead emits one borrow of width one at composed offset one, leaving sibling x.1.1 untouched.

521 4.5 Escaping-borrow place lowering

Reference formation needs a distinct judgment

$$523 \quad C \stackrel{\text{borrow}}{\vdash}_{k,c,m} p \rightarrow C'; (r, D).$$

It follows the same path normalization but always emits a final borrow $(k, c, m, |\tau|, r_b, \delta)$, including when $\delta = 0$. For a dereference it first loads the stored pointer and then borrows the pointed-to range. The returned cleanup records the fresh final tag as a description of its origin, but compiling a reference expression deliberately does not emit that cleanup: the tag escapes in the pointer value stored by the program.

This contrasts with a route borrow. A route tag is used for one store and then retired before the next source-statement boundary. An escaping tag is an observable result and must remain live; the correctness proof extends the source-to-target tag map with its fresh pair.

532 4.6 Expressions and assignment order

533 For an expression of layout τ , pre-lowering returns a store function F . The proof-core cases are:

MIRLite RHS	Pre-destination code	Deferred store $F(r_d)$
n	No code.	$\text{storecNatTy}([\text{dat}(n)], r_d)$.
copy p	Lower p shared; load $ \tau $ cells as $[\tau]$ into fresh value register r_v ; retire its route tags.	$\text{store}_{[\tau]}(r_v, r_d)$.
$\text{ref}(k, c, m, p)$	Use escaping-borrow lowering and retain the resulting tag.	$\text{storePTy}(r_v, r_d)$.

The copied cells are materialized in a register before destination lowering. This is essential when source and destination resolution have observable permission effects: it realizes the source rule’s “evaluate RHS, then resolve destination, then write” order.

For a nonlocal assignment $d := e$, the emitted intervals concatenate as

$\text{ensureRoot}(d); I_{\text{rhs}}; I_{\text{dst}}; F(r_d); \text{cleanup}(D_e); \text{cleanup}(D_d)$.

A local destination is simpler: root preparation directly produces its pointer register, so the compiler performs the RHS pre-work and store through that register. In both cases preparation precedes the RHS. For nonlocal destinations the RHS precedes destination place lowering.

The executable compiler also lowers uninitialized values, exposed-provenance casts, tag-preserving pointer casts, scaled pointer arithmetic, slice retags, heap allocation, deallocation, conditionals, and protector frames. Each translation is defined by an OSEA-IR sequence, not by a constructor-name correspondence. For example, dynamic allocation lowers its length place shared and emits `allocDyn`, while deallocation loads the pointer value before emitting `dealloc`.

4.7 Complete expression lowering

The non-core RHSs use the same pre-code/store split. In the table, lower returns a pointer register r_s and route cleanup D_s ; r_v is a fresh value register. Every cleanup shown next to a pre-instruction is emitted immediately after it, before destination lowering.

MIRLite RHS	Pre-destination code	Deferred store
uninit	No code.	$\text{storec}_{[\tau]}([\text{undef}]^{ \tau }, r_d)$.
exposeAddr p	Lower p shared; emit $r_v := \text{expose}(r_s)$; clean D_s .	$\text{storeNatTy}(r_v, r_d)$.
fromExposed p	Lower p shared; emit $r_v := \text{fromExposed}(r_s)$; clean D_s .	$\text{storePTy}(r_v, r_d)$.
ptrOffset p d	Lower p shared; emit $r_v := \text{offset}(r_s, d \cdot \tau_p)$, where $\Gamma \vdash p : \text{Ptr } \tau_p$; clean D_s .	$\text{storePTy}(r_v, r_d)$.
refSlice(k, c, p)	Lower p shared; emit $r_v := \text{borrowRest}(k, c, r_s)$; clean D_s .	$\text{storePTy}(r_v, r_d)$.
ptrCast p	Lower p shared; no value register. Its route cleanup must remain live until the copy.	$\text{memcpyPTy}(r_d, r_s)$, then clean D_s .

$[\text{undef}]^{|\tau|}$ denotes a list of exactly $|\tau|$ undefined cells. Pointer offsets are expressed by the source in pointee units, so the compiler scales d by the source pointee width before OSEA-IR sees it. A pointer cast is tag-preserving: using borrow would incorrectly mint a new tag, whereas a one-cell `memcpy` performs the required read and write while copying the pointer value unchanged.

The placement of cleanup is semantic. In `exposeAddr`, for example, the route tag is needed only until the RHS reads its pointer-valued cell, so it dies before destination evaluation. In `ptrCast`, the route pointer is the source of the eventual memcpy, so it survives destination evaluation and dies after the store.

4.8 Statements outside the proof core

The compiler is executable on all source statement forms. Their code order is:

Statement	Emitted operational sequence
<code>alloc d (const n)</code>	Prepare the destination root; lower d mutable; emit fresh $r_h := \text{allocN}_{[\tau]}(n)$; store PTy from r_h through d ; retire the destination's route tags.
<code>alloc d (from p)</code>	Prepare and lower d ; lower length place p shared; emit $r_h := \text{allocDyn}_{[\tau]}(r_p)$ and clean the length route; store the heap pointer through d ; retire the destination's route tags.
<code>dealloc p</code>	Lower p shared; load its PTy value into fresh r_v ; retire only route tags; emit <code>dealloc(r_v)</code> .
<code>assignIf p=w d:=e</code>	Lower the discriminant shared and clean its route; emit <code>skipIf</code> whose relative skip equals the dry-run length of the compiled assignment; then emit that assignment.
<code>pushProtectors</code>	Emit <code>pushProt</code> .
<code>popProtectors</code>	Emit <code>popProt</code> .
<code>halt</code>	Emit <code>halt</code> .

For a local allocation destination, root preparation already returns the store pointer, so no separate mutable destination lowering is needed. For a nonlocal destination, the table's order is deliberate: destination resolution occurs before evaluating a dynamic heap length, matching the source allocation statement rather than the ordinary assignment rule.

Conditional lowering uses a dry run only to compute the guarded fragment's length. Both the probe and real compilation start with the same fresh-register counter and local map, so they emit the same relative-register instruction sequence. Only the real pass extends the code map.

4.9 Program compilation and labels

Emission installs a list contiguously at labels $[n_i, n_i + \text{len}(I))$ and then advances n_i . Compiling a source program folds statement compilation from left to right. If every checked statement succeeds, the target program is the final code map; otherwise compilation returns the first compiler error.

For source statement index i , compiling the prefix $P[0..i)$ from C_0 produces a state C_i . Its next label $C_i.n_i$ is the target entry label for statement i . Monotonicity gives two facts needed by simulation:

$$C_i.n_i \leq C_{i+1}.n_i \quad \wedge \quad \forall q < C_i.n_i. C_{i+1}.K(q) = C_i.K(q).$$

Thus compiling later statements cannot change an already recovered fragment. No fixed instruction-per-statement ratio is assumed: the proof locates a source boundary by replaying prefix compilation and lets each simulation case choose the number of target steps in that interval.

4.10 A dereference-lowering example

The field example contains only projections. A small dereference example shows why pointer loads and cleanup are separate. Suppose $p : \text{Ptr } \tau$ is stored in local y , with $L(y) = (r_y, \text{Ptr } \tau)$. Lowering

$\star y$ shared first returns r_y , then chooses fresh r_p and emits

$$r_p := \text{load}_{\text{PTy}(r_y)}.$$

The result is $(r_p, [])$. There is no $\text{die}(r_p, 1)$: the loaded pointer's tag was stored by the source program. For projected place $(\star y).q$ with nonzero offset δ , lowering next emits a compiler borrow from r_p of the final field width and records only that new register in cleanup. A store therefore executes $\text{load}; \text{borrow}; \text{store}; \text{die}$, retiring the route tag without retiring the source pointer loaded from memory.

4.11 Running compilation derivation

Assume compiler entry state

$$C_0 = (N, j, K, L) \quad L(x) = (r_x, \tau_x),$$

where $\tau_x = (\text{Nat}, (\text{Nat}, \text{Nat}))$, K has no code at or above j , $r_f = R(N)$ is fresh, and j is the next label. No allocation is emitted: x already has a local-map entry.

Flattening gives

$$(x.1).0 \rightarrow x.(1.\text{append}(0)) \quad \text{off}(1.\text{append}(0)) = 1 \quad |\text{Nat}| = 1.$$

The constant has no pre-code. Destination lowering is mutable, uses r_x as the base register, and emits one final-width borrow. The store function and cleanup then determine the rest. Hence

$$C_0 \vdash x.1.0 := 42 \rightarrow C_3,$$

where C_3 has $n_r = N + 1$, $n_l = j + 3$, the same local map, and

$K_3(j)$	$r_f := \text{borrow}(\text{mutable}, \text{false}, [], 1, r_x, 1)$
$K_3(j + 1)$	$\text{storec}_{\text{NatTy}}([\text{dat}(42)], r_f)$
$K_3(j + 2)$	$\text{die}(r_f, 1)$

Every number is now derived: path composition gives offset one; the selected layout gives borrow length one; erasure gives NatTy ; the constant gives $\text{dat}(42)$; and reversing the singleton cleanup gives the last instruction. This is exactly the OSEA-IR execution derived in the previous section.

Formalization of compiler state growth, checked lowering, and program compilation: `src/obseq3/compile.lean`.
Flattening and its semantic preservation are in `src/obseq3/proof/spine.lean`.

5 Compiler correctness

Correctness is a forward simulation of successful executions. Literal state equality is impossible: the target has registers and introduces route tags (Definition 5.1). The relation instead compares source-observable memory, local pointers, and permission stacks at source-statement boundaries. This section defines the relation (Definition 5.2 to Definition 5.6), states the two lemmas the proof turns on (Lemma 5.7 and Lemma 5.8), and then states the simulation theorems (Theorem 5.9 to Corollary 5.12).

5.1 Renamings and value simulation

Definition 5.2 (Renamings) *An address renaming is a partial map $\rho_a : \mathbb{N} \rightarrow \text{option}(\mathbb{N})$ and a tag renaming is a partial map $\rho_t : \text{Tag} \rightarrow \text{option}(\text{Tag})$. A renaming ρ' extends ρ , written $\rho \subseteq \rho'$, when ρ' agrees with ρ wherever ρ is defined.*

Both maps are partial because future allocations and tags have not yet been created. Address renaming turns out to be the identity on its domain, for a reason that combines a property of the two machines with a property of the compiler. Each machine allocates with a deterministic bump allocator that never reuses addresses, and both start at the same watermark. The compiler emits exactly one target allocation, of the same size, for each source allocation, and in the same order. Hence the two watermarks stay equal at every statement boundary, and each fresh block receives the same base on both sides:

$$\rho_{a(a)} = a' \rightarrow a' = a.$$

The notation remains useful because allocation extends the domain by a whole fresh block. Tags genuinely require renaming: route borrows advance the target tag counter, so a later source and target ref may mint different numeric tags.

Definition 5.3 (Value simulation) *Source cell value v is simulated by target value v' under (ρ_a, ρ_t) , written $v \approx v'$, when one of the following holds.*

- $v = \text{word}(w)$ and $v' = \text{dat}(w)$.
- $v = \text{ptr}(b, o, s, t)$, $v' = \text{ptr}(b', o, s, t')$, $\rho_a(b) = b'$, $\rho_t(t) = t'$, and every address in $[b, b + s)$ is in the domain of ρ_a .
- $v = \text{undef}$, and v' is arbitrary.

Source undef relates to any target value because a successful core source run cannot observe the missing information.

Definition 5.4 (Memory simulation) *Source memory μ_s is simulated by target memory μ_t under (ρ_a, ρ_t) when, for every address a at which μ_s has a cell v , $\rho_a(a)$ is defined and $v \approx \mu_t(\rho_a(a))$.*

The relation is forward only: every present source cell has a related target cell at its mapped address, and extra unobservable target cells are permitted.

5.2 Permission simulation

Definition 5.5 (Permission simulation) *Let items, stacks, and states be those of the Stacked Borrows instance of Section 2.3. Under ρ_t :*

- Two items are related when they have the same constructor and ρ_t maps the source tag to the target tag; raw-pointer items additionally carry the same mutability flag.
- Two stacks are related position by position, so their lengths agree.
- Two stack maps are related when, for every address a , either both lack an entry at a or their entries at a are related stacks.
- Two protector frame lists are related as lists of related tag lists, and two exposed-tag lists are related as tag lists.

Permission states Π_s and Π_t are related, written $\Pi_s \approx \Pi_t$, when their stack maps, protector frames, and exposed lists are related and $\Pi_s.\text{NextTag} \leq \Pi_t.\text{NextTag}$.

Stack-map comparison is by address lookup, not by the list position used to implement the finite map. Disjoint operations may reorder that representation while leaving all address observations the same.

The counter condition is an inequality rather than an equality. Suppose a route borrow mints route tag u at the target counter. Inside the compiled fragment the target stacks contain an extra item, so the boundary relation is not expected to hold. The store uses u and `die` removes its item. The target counter remains advanced, which is why a later pair of corresponding source and target tags may have different numeric values.

When a source-visible reference is formed, both machines mint a tag. If they produce t_s and t_t , the proof extends

$$\rho_{t'} = \rho_{t[t_s \mapsto t_t]}.$$

Because both fresh tags are minted at their machine's counter, and the boundary invariant below keeps every mapped tag strictly below both counters, neither fresh tag is already in the old map's domain or range, so injectivity is preserved. By contrast, a route borrow never extends ρ_t , since there is no source tag to use as a key.

5.3 The boundary invariant

Definition 5.6 (Boundary invariant) *Let C_0 be a compiler state and P a source program. Source state $S = (i, E, \mu_s, \Pi_s)$ and target state $T = (j, R, \mu_t, \Pi_t)$ satisfy the boundary invariant under (ρ_a, ρ_t) , written $S \stackrel{C_0, P}{\approx}_{\rho_a, \rho_t} T$, when there is a compiler state C_i such that the following ten clauses hold.*

- (I1) Control alignment. *Compiling the prefix $P[0..i]$ from C_0 succeeds with state C_i , and $j = C_i.n_l$.*
- (I2) Bound locals. *If $E(\ell) = (a, t)$ for $\ell : \tau$, then $L_{i(\ell)} = (r, \tau)$ and $R(r) = (\text{PTy}, [\text{ptr}(a, 0, |\tau|, t')])$ with $\rho_{a(a)} = a$ and $\rho_{t(\ell)} = t'$, and every address of $[a, a + |\tau|)$ is in the domain of ρ_a .*
- (I3) Source memory. μ_s is simulated by μ_t (Definition 5.4).
- (I4) Permissions. $\Pi_s \approx \Pi_t$ (Definition 5.5).
- (I5) Address identity. ρ_a is the identity wherever defined.
- (I6) Tag-map shape. ρ_t is injective and maps the wildcard tag to itself.
- (I7) Tag bounds. *Every source tag in the domain of ρ_t lies below $\Pi_s.\text{NextTag}$, and every tag in its range lies below $\Pi_t.\text{NextTag}$.*
- (I8) Allocator lockstep. *The source and target bump-allocation watermarks are equal.*
- (I9) Unbound locals. *If E has no binding for ℓ , then L_i has no entry for ℓ .*
- (I10) Register freshness. *Every register in the range of L_i is below $C_i.n_r$.*

The invariant is required only at source-statement boundaries and is deliberately strict about stack shape there. The compiled fragment may push a route tag's item inside a fragment, but `die` removes it before the invariant is re-established. Numeric next-tag counters may still differ, which is why tags are renamed and the target counter need only be greater.

Each auxiliary clause discharges a specific obligation in the step proof.

- (I1) with Lemma 5.7: the instruction fetched at j belongs to the fragment compiled for statement i , and compiling later statements has not overwritten it.
- (I2) with (I9): exactly one allocation regime applies to a destination root. Either a corresponding target pointer register exists, or both sides allocate.
- (I3) with (I5): a successful source read yields related target cells at the same concrete range, and writes update corresponding addresses.

- (I4) with (I6): related source-visible operations accept corresponding tags and leave related stacks after renaming.
- (I6) with (I7): a newly minted source and target tag pair extends ρ_t without overwriting or aliasing an old pair.
- (I8) with the block-domain part of (I2): fresh roots receive the same base on both sides, which is what makes the identity renaming of Definition 5.2 sound, and every cell of a multi-cell or zero-sized binding has the domain fact needed by later pointer simulation.
- (I10): a fresh register cannot collide with a register that the target uses as a source-local pointer.

Dropping any clause leaves a concrete compiler case underdetermined. Memory agreement alone cannot show that a fresh load register does not overwrite r_x , and equal watermarks alone cannot show that an unbound source local corresponds to a target fragment containing alloc.

5.4 Two lemmas

The step proof relies on a fact about the compiler and a fact about the permission model.

Lemma 5.7 (Prefix monotonicity) *Let C_i and C_{i+1} be the states obtained by compiling $P[0..i]$ and $P[0..i+1]$ from C_0 . Then $C_i.n_l \leq C_{i+1}.n_l$, and $C_{i+1}.K(q') = C_i.K(q')$ for every label $q' < C_i.n_l$.*

This is the monotonicity property of Section 4.9 specialized to consecutive prefixes. It lets the proof locate a source boundary by replaying prefix compilation, with no fixed instruction-per-statement ratio.

Lemma 5.8 (Route borrow cancellation) *Let Π be a permission state whose next tag is not the wildcard and is not protected in any frame. Suppose $\text{ref}(\Pi, a, n, t, \text{mutable}, \text{false}, []) = (\Pi_1, u)$. Then there are Π_2, Π_3 , and Π_{acc} with*

$$\text{useMut}(\Pi_1, a, n, u) = \Pi_2, \quad \text{die}(\Pi_2, a, n, u) = \Pi_3, \quad \text{useMut}(\Pi, a, n, t) = \Pi_{\text{acc}},$$

such that Π_3 and Π_{acc} have the same stack map, protector frames, and exposed list, and $\Pi_{\text{acc}}.\text{NextTag} \leq \Pi_3.\text{NextTag}$.

Lemma 5.8 is the semantic content of the compiled write pattern borrow; store; die. The three target events have exactly the stack effect of the single source useMut through the parent tag, and differ only in the tag counter. That difference is absorbed by the inequality in Definition 5.5. The two side conditions are invariants of every reachable permission state: tags are minted from one upward, and protector frames only ever contain already-minted tags.

5.5 Simulation theorems

Let `srcStep` fetch the source statement at the current PC and execute it, treating `halt` and a missing statement as successful fixed points. The *core fragment* consists of `halt` and assignments whose RHS is a constant, a copy, or a reference with any retag kind k , protector flag c , and mask m . Here “any kind” includes shared, mutable, both raw pointer kinds, and two-phase. A program is *core* when every statement in it is.

Theorem 5.9 (One-step forward simulation) *Let P be a core program, C_0 any compiler state, and Q the result of compiling P from C_0 . If*

$$S \underset{\rho_a, \rho_t}{\overset{C_0, P}{\approx}} T \quad \text{and} \quad \text{srcStep}(P, S) = \text{ok}(S'),$$

then there exist $\rho_{a'} \supseteq \rho_a$, $\rho_{t'} \supseteq \rho_t$, a target state T' , and n_t such that

$$Q \vdash T \xrightarrow{n_t} T' \quad \text{and} \quad S' \underset{\rho_{a'}, \rho_{t'}}{\overset{C_0, P}{\approx}} T'.$$

The compiler state in the final invariant is still C_0 . Compilation is not executing alongside the target. The invariant internally recompiles the appropriate prefix to identify the next target label. Only the renamings grow during simulation. The target step count n_t is existential because fragments have different lengths. For `halt` or a missing source statement, $n_t = 0$.

Proof sketch. Case on the source statement at i . For `halt` or a missing statement the source state is unchanged, and the invariant is re-established with $n_t = 0$ and the same renamings. For an assignment $d := e$, (I1) and Lemma 5.7 recover the emitted label interval for statement i , and the proof executes it phase by phase, following the operational order of Section 2.5 and the compiler order of Section 4.6:

source RHS	→	target pre-code	→	destination store	and	→	boundary invariant
------------	---	-----------------	---	----------------------	-----	---	--------------------

RHS. For a constant, source and target stores will write related singleton values. For a copy, (I3) and (I5) give related reads producing cell lists of equal length; the target register holds that list across destination resolution, which is what makes the source's evaluate-then-resolve order simulable. For an escaping reference, (I4) gives corresponding `ref` events that mint a fresh tag pair (t_s, t_t) ; (I6) and (I7) let ρ_t extend by that pair, after which the two pointer values are related by Definition 5.3.

Destination. If the root is already bound, (I2) identifies the target register holding its pointer. If it is unbound, (I9) guarantees the compiled fragment contains the matching `alloc`; (I8) gives both allocators the same fresh base; corresponding `own` events mint a tag pair; ρ_a extends by the identity on the whole block and ρ_t by the tag pair. Typed paths give exact offsets and widths. A dereference destination loads a pointer related by (I3). (I10) ensures the loads and route borrows do not overwrite a register named by L_i .

Store and cleanup. For a nonzero-offset destination the fragment ends in `borrow`; `store`; `die`. Lemma 5.8 shows the three target permission events equal the single source `useMut` up to the tag counter, under the *unchanged* ρ_t . Related writes `restore` (I3). The final target PC is $C_{i+1}.n_t$, which re-establishes (I1) for $i + 1$; the remaining clauses are preserved or extended as described. $\square$

Theorem 5.10 (Forward simulation of finite runs) *Let P be a core program, C_{init} the compiler state with empty code, counters zero, and an empty local map, and Q the result of compiling P from C_{init} . If*

$$S \underset{\rho_a, \rho_t}{\overset{C_{\text{init}}, P}{\approx}} T \quad \text{and} \quad \text{runS}^n(P, S) = \text{ok}(S'),$$

then there exist $\rho_{a'} \supseteq \rho_a$, $\rho_{t'} \supseteq \rho_t$, a target state T' , and n_t such that

$$\text{runT}^n(Q, T) = \text{ok}(T') \quad \text{and} \quad S' \underset{\rho_{a'}, \rho_{t'}}{\overset{C_{\text{init}}, P}{\approx}} T'.$$

Proof sketch. Induction on n_s . Each successful source statement uses Theorem 5.9, and the induction hypothesis simulates the remainder. Extension of renamings is transitive, so the two extensions compose; target step counts compose by addition. When the source encounters halt or the end of the program, its fuelled semantics stops and the proof chooses zero further target steps. $\square$

Theorem 5.10 takes the initial invariant as a premise; it does not hide initialization inside compilation. The premise is discharged for closed programs by the following lemma.

Lemma 5.11 (Initial relation) *Let S_{init} and T_{init} be the source and target states with empty environment, register file, and memory, PC zero, both bump allocators at watermark zero, and both permission states initialized. Let ρ_a be nowhere defined and let ρ_t map only the wildcard tag to itself. Then $S_{\text{init}} \overset{C_{\text{init}}, P}{\approx}_{\rho_a, \rho_t} T_{\text{init}}$ for every P .*

Proof sketch. Each clause has a direct witness. Compiling the empty prefix returns C_{init} , whose next label is zero, giving (I1). (I2) and (I3) are vacuous. The two initial permission states have identical stack maps, frames, and exposed lists and compatible counters, giving (I4). (I5) and (I8) are immediate. The wildcard-only map is injective and lies below the initialized counters, giving (I6) and (I7). The empty local map gives (I9) and (I10). $\square$

Corollary 5.12 (Closed programs) *Let P be a core program and Q its compilation from C_{init} . If $\text{runS}^{n_s}(P, S_{\text{init}}) = \text{ok}(S')$, then there exist $\rho_{a'}$, $\rho_{t'}$, T' , and n_t with $\text{runT}^{n_t}(Q, T_{\text{init}}) = \text{ok}(T')$ and $S' \overset{C_{\text{init}}, P}{\approx}_{\rho_{a'}, \rho_{t'}} T'$.*

Proof sketch. Theorem 5.10 with Lemma 5.11. $\square$

These results are preservation of successful finite executions. They are not backward simulation, divergence preservation, or an equivalence between source and target error strings. Their observable consequences are the memory and permission clauses of the final invariant, made explicit in Corollary 5.14 below.

5.6 The running example as a simulation step

Example 5.13 Return to $x.1.0 := 42$. At the source-statement boundary assume

$$\rho_a(100) = 100 \quad \rho_t(t_x) = \hat{t}_x$$

and let the invariant supply

$$\begin{aligned} E(x) &= (100, t_x) \quad R(r_x) = (\text{PTy}, [\text{ptr}(100, 0, 3, \hat{t}_x)]) \\ \mu_s[100..103] &= [\text{word}(5), \text{word}(7), \text{word}(9)] \quad \mu_t[100..103] = [\text{dat}(5), \text{dat}(7), \text{dat}(9)]. \end{aligned}$$

The source derivation of Section 2 resolves the flattened path to address 101 and performs $\text{useMut}(\Pi_s, 101, 1, t_x) = \Pi_{s'}$. The compiler derivation of Section 4 supplies the exact three-instruction fragment, and the target derivation of Section 3 is

$$\begin{aligned} \text{ref}(\Pi_t, 101, 1, \hat{t}_x, \text{mutable}, \text{false}, []) &= (\Pi_1, u) \\ \text{useMut}(\Pi_1, 101, 1, u) &= \Pi_2 \quad \text{die}(\Pi_2, 101, 1, u) = \Pi_{t'}. \end{aligned}$$

Lemma 5.8 removes the route tag u 's item and recovers (I4) under the unchanged ρ_t : u has no source counterpart and never escapes. Both memories update only address 101, where source word 42 relates to target $\text{dat}(42)$, so (I3) is restored and addresses 100 and 102 are framed. (I1) advances from source PC i to $i + 1$ and from target label j to the next prefix label $j + 3$. Therefore

$$(i + 1, E, \mu_s[101 \mapsto 42], \Pi_{s'}) \overset{C_{0,P}}{\underset{\rho_a, \rho_t}{\approx}} (j + 3, R', \mu_t[101 \mapsto \text{dat}(42)], \Pi_{t'}).$$

Flattening is semantically load-bearing here. A borrow of intermediate width two would operate on address 102 as well, so Lemma 5.8 would not simulate the source's one-cell write in the presence of a sibling borrow. The compiled path must use the composed offset and final width.

The boundary assumed in Example 5.13 is itself produced by an earlier application of Theorem 5.9. Suppose the first statement assigns a constant to an unbound local $x : \tau$. MIRLite root preparation allocates $|\tau|$ cells and obtains owning tag t_s ; target preparation emits $r_x := \text{alloc}_{|\tau|}$, allocates the same base by (I8), and obtains t_t . The proof extends ρ_a by the identity on the entire block, extends ρ_t with $t_s \mapsto t_t$, and extends the local map with $x \mapsto (r_x, \tau)$. The subsequent store establishes (I3) for the initialized cells. The environment, register, memory, and tag mapping assumed in Example 5.13 are precisely these facts, not an unexplained alternate initialization.

The four formal views of $x.1.0 := 42$ thus line up. MIRLite's typed path has input layout τ_x , selected layout Nat , composed offset one, and width one; source resolution returns base 100, address 101, size three, and tag t_x , and the statement performs one mutable permission use and one cell write. OSEA-IR begins with an erased PTy pointer in r_x , which the compiler's local map connects to $x : \tau_x$. Flattening prevents an intermediate two-cell borrow, so place lowering creates exactly one fresh mutable tag for address 101; constant lowering supplies $\text{dat}(42)$ and NatTy ; cleanup supplies the matching one-cell die. At the boundary, address identity relates the updated cell, memory simulation frames its neighbours, permission simulation cancels the route tag without adding it to ρ_t , and prefix compilation places the target at $j + 3$ exactly when the source reaches $i + 1$:

typed source range $\rightarrow$ flattened target route $\rightarrow$ explicit events $\rightarrow$ boundary simulation.

The example explains why the compiler may add a borrow, why that borrow must have final-field width, why it must be retired, and which invariant clauses make the extra events unobservable at the next source boundary.

5.7 Observable consequences

Corollary 5.14 (Observations at a boundary) *If $S \stackrel{C_0, P}{\approx}_{\rho_a, \rho_t} T$, then:*

- if $\mu_{s(a)} = \text{word}(w)$, then $\mu_{t(a)} = \text{dat}(w)$;
- if $\mu_{s(a)} = \text{ptr}(b, o, s, t)$, then $\mu_{t(a)} = \text{ptr}(b, o, s, t')$ with $\rho_{t(t)} = t'$ and the referent range $[b, b + s)$ mapped;
- if $E(\ell) = (a, t)$ for $\ell : \tau$, then the register $L_{i(\ell)}$ holds PTy and exactly one pointer with base a , offset zero, size $|\tau|$, and tag $\rho_{t(t)}$;
- the target stack at every address has the same length and item constructors as the source stack, with every source tag renamed by ρ_t ;
- the target PC is the next label after compiling exactly $P[0..i)$.

Proof sketch. Clauses (I3) and (I5) give the first two items, (I2) the third, (I4) the fourth, and (I1) the fifth. $\square$

Several asymmetries are intentional. The target may contain extra registers, because value temporaries remain after use. Source undef refines any target cell, so the memory relation has no reverse-domain guarantee. Address maps are partial even though defined pairs are identities. Tag counters need not be equal, and numeric tag equality is not observable. Finally, the invariant is required only at source-statement boundaries; intermediate target states may contain live route tags.

These choices are the minimum abstraction needed by the actual generated code. Strengthening to literal register equality would reject harmless temporaries, while weakening permission stacks to ignore arbitrary target items would no longer justify later accesses.

5.8 Extending the theorem to the executable surface

The remaining compiler cases are specified and executable, but adding them to Theorem 5.9 requires semantic simulation arguments. The needed work is not “support the constructor”; it is to preserve the ten clauses of Definition 5.6 through each emitted sequence:

Source form	Missing simulation obligation
uninit	Relate the source’s $ \tau $ undefined cells to the target’s explicit undef list after corresponding mutable writes; re-establish (I3) without demanding information from those cells.
exposeAddr	Transport the outer pointer-cell read, show the stored related tags are exposed on both sides, and relate the resulting concrete address words using (I5).
fromExposed	Relate allocation-table resolution and wildcard-tagged pointers, including the degenerate pointer returned for an address outside all recorded allocations.
ptrCast / ptrOffset	For casts, prove the target memcpy realizes the source pointer-value copy without retagging. For offsets, transport the read and show scaling by the pointee width $ \tau_p $ agrees with source cell arithmetic and failure bounds.

refSlice	Relate the runtime remaining lengths of stored pointers, transport the two-stage read-then-retag event order, and extend ρ_t at the escaping tag pair.
alloc / dealloc	For allocation, synchronize a runtime or constant length and extend both renamings over the fresh heap block. For deallocation, relate offset-zero checks, permission retirement, and removal of the same memory range.
assignIf	Relate the event-free discriminant lookup and show both branch outcomes land at the prefix label following either the skipped or executed assignment fragment.
Protector frames	Show push and successful pop preserve the nested tag-list relation of Definition 5.5, including protected tags introduced by source-visible references.

Several obligations reuse existing core machinery, but none follows merely from successful differential tests. In particular, `assignIf` needs a control argument about relative skip length, while `fromExposed` needs a value relation for wildcard provenance. Listing these obligations keeps the stated theorem boundary aligned with what is actually proved.

5.9 Scope and mechanization status

PRECISE CLAIM The mechanized theorems are forward simulations for successful finite runs of core programs: `halt` and assignments from constants, copies, and references. The executable compiler covers the larger syntax described above, but those forms are outside Theorem 5.9 until their simulation cases are added.

Every numbered definition and result of this section is a Lean 4 declaration in `src/obseq3/proof/`:

This paper	Lean declaration	File
Definition 5.3	<code>MemValSim</code>	<code>common.lean</code>
Definition 5.4	<code>SourceMemSim</code>	<code>common.lean</code>
Definition 5.5	<code>PermSim</code>	<code>common.lean</code>
Definition 5.6	<code>CompilerInv</code>	<code>common.lean</code>
Lemma 5.8	<code>sb_ref_use_die_cancels</code>	<code>keystone.lean</code>
Theorem 5.9	<code>CompilerInv_step</code>	<code>compiler.lean</code>
Theorem 5.10	<code>compile_correct</code>	<code>compiler.lean</code>
Lemma 5.11	<code>CompilerInv_initial</code>	<code>compiler.lean</code>
Corollary 5.12	<code>compile_correct_from_initial</code>	<code>compiler.lean</code>

None of these contains an admitted goal. A checked audit prints the axioms that Theorem 5.10 and Corollary 5.12 depend on and fails if that set differs in either direction from a pinned whitelist. The whitelist contains exactly the three standard Lean axioms, propositional extensionality, choice, and quotient soundness, and no `sorryAx`. The last admitted case, the reference-assignment leaves, was closed on 2026-08-31.

The broader executable compiler is validated by testing rather than proof: a compiler witness corpus, a ULLBC conformance corpus checked against Miri verdicts, and a differential run that compiles each conformance program and requires the same verdict from both machines. Those

944 tests cover the constructs listed in the previous subsection, but only a discharged simulation case
945 moves a construct inside Theorem 5.9.

946 Invariant and simulation vocabulary: `src/obseq3/proof/common.lean`. Statement and whole-program forward sim-
947 ulations: `src/obseq3/proof/compiler.lean`.